7.2 CI/CD 파이프라인 분석

개요

Composable Finance 프로젝트는 GitHub Actions와 Nix를 기반으로 한 현대적이고 강력한 CI/CD 파이프라인을 구축하여 코드 품질, 보안, 그리고 배포 자동화를 보장합니다. 이 파이프라인은 다단계 검증, 병렬 처리, 그리고 재현 가능한 빌드를 통해 높은 신뢰성을 제공합니다.

7.2.1 GitHub Actions 워크플로우 구조

주요 워크플로우 파일

프로젝트의 CI/CD는 다음과 같은 워크플로우 파일들로 구성됩니다:

```
.github/workflows/
├── pr-workflow.yml // 메인 PR 검증 워크플로우
├── pr-workflow-check.yaml // PR 워크플로우 체크
├── pr-workflow-fork.yaml // 포크 PR 처리
├── benchmark.yml // 벤치마크 자동화
├── release-artifacts.yml // 릴리스 아티팩트 생성
├── release-docs.yml // 문서 배포
└── maintainance.yaml // 유지보수 작업
```

메인 PR 워크플로우 (`pr-workflow.yml`)

메인 워크플로우는 다음과 같은 구조로 구성됩니다:

```
name: "pr-workflow"
concurrency:
group: ${{ github.workflow }}-pr-workflow-${{ github.event.pull_request.number |
cancel-in-progress: true


on:
workflow_call:
inputs:
github_event_name: { required: true, type: string }
github_event_pull_request_head_repo_id: { required: true, type: number }
flow: { required: true, type: string }
sha_to_check: { required: true, type: string }
```

7.2.2 빌드 과정 분석

의존성 빌드 단계

CI/CD 파이프라인은 계층적 빌드 구조를 사용합니다:

```
build-all-deps-packages:
name: build-all-deps-packages
runs-on: [x86_64-linux-32C-128GB-2TB]
```

```
steps:
- name: Set up Cachix
uses: cachix/cachix-action@586bf280495080c5a6d4868237ad28a860e4b309
with:
authToken: "${{ secrets.CACHIX_AUTH_TOKEN }}"
name: composable
- name: build-all-deps-packages
run: nix -- build .#all-deps
```

출력 패키지 빌드

의존성 빌드 완료 후 실제 출력 패키지들을 빌드합니다:

```
build-all-outputs-packages:
needs: [build-all-deps-packages]
runs-on: [x86_64-linux-32C-128GB-2TB]
steps:
- name: Build all packages
run: nix -- build .#all-outputs
```

검증 패키지 빌드

최종적으로 모든 검증 도구와 테스트를 실행합니다:

```
build-all-checks-packages:
```

```
needs: [build-all-outputs-packages]

runs-on: [x86_64-linux-32C-128GB-2TB]

steps:

- name: Build all packages

run: nix -- build .#all-checks
```

7.2.3 테스트 자동화

Nix Flake 검증

프로젝트 구성의 유효성을 검증합니다:

```
nix-flake-check:

runs-on: [ubuntu-latest-m]

steps:

- uses: cachix/install-nix-action@v20

- uses: DeterminateSystems/magic-nix-cache-action@main

- run: nix run .#nix-flake-check --accept-flake-config
```

Nix Flake 검증 스크립트:

```
# flake/check.nix에서 정의된 검증 로직

NIXPKGS_ALLOW_BROKEN=1  nix  flake  check \

--keep-going \

--no-build \
```

```
--allow-import-from-derivation \
--accept-flake-config \
--system "${system}" \
--fallback \
--impure
```

의존성 보안 검토

보안 취약점을 자동으로 검사합니다:

```
dependency-review:
steps:
- name: dependency-review
uses: actions/dependency-review-action@v3
with:
allow-ghsas: GHSA-pfrx-2q88-qq97, GHSA-w5p7-h5w8-2hfq, ...
```

Mantis E2E 테스트

통합 테스트를 자동으로 실행합니다:

```
mantis-e2e:
needs: [build-all-checks-packages]
runs-on: [ubuntu-latest-m]
steps:
```

```
- name: Devnet integration tests

run: nix run .#mantis-e2e --accept-flake-config --impure
```

7.2.4 코드 품질 검사

Clippy 린팅

Rust 코드 품질을 자동으로 검사합니다:

```
# flake/ci.nix에서 정의

cargo-clippy-check = pkgs.runCommand "cargo-clippy-check" {

buildInputs = [ rustToolchain ];

} ''

cargo clippy --all-targets --all-features -- -D warnings

'';
```

포맷팅 검사

코드 스타일 일관성을 검증합니다:

```
cargo-fmt-check = pkgs.runCommand "cargo-fmt-check" {

buildInputs = [ rustToolchain ];

} ''

cargo fmt --all -- --check

'';
```

Deny 검사

라이선스 및 보안 정책을 검증합니다:

```
cargo-deny-check = pkgs.runCommand "cargo-deny-check" {
buildInputs = [ cargo-deny ];
} ''
cargo deny check
'';
```

7.2.5 벤치마크 자동화

벤치마크 워크플로우

성능 회귀를 방지하기 위한 자동 벤치마크 시스템:

```
# .github/workflows/benchmark.yml
update-benchmarks-weights:
runs-on: [self-hosted, bmark]
strategy:
matrix:
runtime: [picasso, composable]
if: contains(github.event.pull_request.labels.*.name, 'needs-benchmarks')
steps:
- name: Generate weights
```

```
run: nix run .#benchmarks-generate-${{ matrix.runtime }} -L
```

가중치 자동 업데이트

벤치마크 결과를 기반으로 런타임 가중치를 자동 업데이트합니다:

```
- name: Push generated weights
uses: EndBug/add-and-commit@v9
with:
add: "code/parachain/runtime"
message: "chore(benchmarking): update ${{ matrix.runtime }} benchmarking weights
committer_name: ${{ steps.gpg.outputs.name }}
committer_email: ${{ steps.gpg.outputs.email }}
```

벤치마크 실행 조건

벤치마크는 다음 조건에서 실행됩니다:

1. **라벨 기반 트리거**: `needs-benchmarks` 라벨이 있는 PR
2. **스케줄 기반**: 매월 1일과 15일 자동 실행
3. **수동 트리거**: `workflow_dispatch` 이벤트

7.2.6 Nix 기반 개발 환경

재현 가능한 빌드 시스템

Nix를 통해 완전히 재현 가능한 빌드 환경을 제공합니다:

```
# flake.nix 주요 구성
{
inputs = {
nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
rust-overlay.url = "github:oxalica/rust-overlay";
crane.url = "github:ipetkov/crane";
};
outputs = { self, nixpkgs, ... }: {
packages = {
composable-node = /* 노드 빌드 정의 */;
devnet-image = /* 개발 네트워크 이미지 */;
};
};
}
```

캐시 최적화

Cachix를 통한 빌드 캐시 최적화:

```yaml
- uses: cachix/cachix-action@master
with:
authToken: "${{ secrets.CACHIX_AUTH_TOKEN }}"
name: composable
skipAddingSubstituter: false
```

```
skipPush: false
```

개발 셸 환경

일관된 개발 환경을 제공합니다:

```
devShells.default = pkgs.mkShell {
buildInputs = with pkgs; [
rust-toolchain
nodejs
docker
# 기타 개발 도구들
];
shellHook = ''
echo "Composable Finance 개발 환경에 오신 것을 환영합니다!"
'';
};
```

7.2.7 Docker 이미지 빌드 및 배포

자동 이미지 빌드

성공적인 빌드 후 Docker 이미지를 자동 생성합니다:

```
push-docker-images:
```

```
needs: [draft-release-check]

steps:

- name: Publish cmc-api to docker hub

uses: "./.github/templates/docker-publish"

with:

image_path: result/docker-image-cmc-api.tar.gz

username: ${{ secrets.DOCKER_HUB_USERNAME }}

password: ${{ secrets.DOCKER_HUB_ACCESS_TOKEN }}

name: cmc-api

artifact: cmc-api:latest
```

다중 이미지 지원

다양한 용도의 Docker 이미지를 생성합니다:

1. **cmc-api**: API 서버 이미지
2. **devnet-xc**: 크로스체인 개발 네트워크
3. **hyperspace-***: IBC 릴레이어 이미지들

7.2.8 릴리스 자동화

릴리스 아티팩트 생성

태그 기반 자동 릴리스:

```
# .github/workflows/release-artifacts.yml

on:
```

```yaml
push:
tags:
- 'release-v[0-9]+\.[0-9]+\.[0-9]+'

steps:
- run: |
nix run .#generate-release-artifacts --print-build-logs
COMPOSABLE_VERSION=$(nix eval --raw .#composable-node.version)
docker tag composable:latest "composablefi/composable:v$COMPOSABLE_VERSION"
```

릴리스 노트 자동 생성

GitHub의 자동 릴리스 노트 기능을 활용합니다:

```yaml
- name: Release artifacts
uses: softprops/action-gh-release@v1
with:
draft: false
prerelease: true
generate_release_notes: true
body_path: release-artifacts/release.txt
```

7.2.9 보안 및 권한 관리

시크릿 관리

민감한 정보는 GitHub Secrets로 관리됩니다:

```
secrets:
NIXBUILD_TOKEN: { required: true }
CACHIX_AUTH_TOKEN: { required: true }
DOCKER_HUB_USERNAME: { required: true }
DOCKER_HUB_ACCESS_TOKEN: { required: true }
GLADOS_GPG_PRIVATE_KEY: { required: true }
```

권한 제어

최소 권한 원칙을 적용합니다:

```
permissions:
pull-requests: write
contents: write
```

포크 PR 처리

외부 기여자의 포크 PR을 안전하게 처리합니다:

```
privilege-check:
if: ${{ inputs.flow == 'push' ||
inputs.github_event_name == 'merge_group' ||
```

```
(inputs.github_event_name == 'pull_request_target' &&
inputs.github_event_pull_request_head_repo_id != 383289760) }}
```

7.2.10 성능 최적화

병렬 처리

여러 작업을 병렬로 실행하여 CI 시간을 단축합니다:

```
strategy:
matrix:
runtime: [picasso, composable]
```

캐시 활용

빌드 캐시를 적극 활용하여 빌드 시간을 단축합니다:

```
- uses: DeterminateSystems/magic-nix-cache-action@main
- uses: cachix/cachix-action@master
```

리소스 최적화

적절한 러너 크기를 선택하여 비용과 성능을 균형있게 관리합니다:

```
runs-on:
- x86_64-linux-32C-128GB-2TB  # 대용량 빌드용
- ubuntu-latest-m  # 일반 작업용
```

```
- self-hosted  # 벤치마크용
```

이러한 포괄적인 CI/CD 파이프라인을 통해 Composable Finance 프로젝트는 높은 코드 품질, 보안성, 그리고 배포 안정성을 보장하며, 복잡한 블록체인 인프라의 지속적인 통합과 배포를 효율적으로 관리합니다.